

Sub-module 1

INTRODUCTION TO

RISC-V Architecture

A complete beginner's guide to the open-source Instruction Set Architecture

From basic concepts to pipeline stages and extensions

What We'll Cover

01

What is RISC-V ISA and why PicoRV32 for our SoC Subsystem?

02

32 registers (x0-x31) — their role in firmware execution

03

Instruction formats — how rom.hex words map to RISC-V opcodes

04

PicoRV32 native memory interface (mem_valid / mem_ready)

05

Lab: make pico — run PicoRV32 standalone simulation

06

Waveform: observe instruction fetch cascade in GTKWave

What is an Instruction Set Architecture (ISA)?

Analogy:

Think of an ISA like a restaurant menu. The menu (ISA) lists all dishes (instructions) the kitchen (hardware) can make. Software is the customer who orders from the menu — it doesn't need to know HOW the kitchen works, only WHAT it can order.

What it defines

- The list of valid CPU instructions
- How memory is addressed
- How registers are used
- How the OS talks to hardware

Why it matters

- Software written for one ISA runs on ALL chips that implement it
- Separates hardware design from software design
- Enables a huge ecosystem of tools & compilers

Examples

- x86 — Used in most PCs
- ARM — Used in phones & tablets
- MIPS — Used in routers
- RISC-V — Open source, growing fast

RISC vs CISC — Two Design Philosophies

Feature	RISC (Reduced Instruction Set)	CISC (Complex Instruction Set)
Instruction count	Few, simple instructions	Many, complex instructions
Instruction length	Fixed (e.g. 32-bit)	Variable length
Clock cycles/instr	Usually 1 cycle each	1 to many cycles
Memory access	Load/store model only	Any instruction can access memory
Hardware	Simple, smaller chip area	Complex decoder circuitry
Examples	ARM, MIPS, RISC-V	x86, x86-64 (Intel/AMD)
Best for	Embedded, mobile, efficiency	Legacy desktop apps, x86 ecosystem

RISC-V Overview

Open-Source ISA — Origin & Ecosystem

Born	UC Berkeley, 2010 — free, open, royalty-free ISA
Base ISA	RV32I / RV64I — fixed 32-bit instructions
Modular	Optional extensions: M (mul), F (float), V (vector)...
Governed	RISC-V International — non-profit standards body
This Project	PicoRV32 — minimal RV32I in ~3100 lines of Verilog

PicoRV32 — The CPU in This Project

File: rtl/picorv32.v (~3100 lines of pure Verilog RTL) • ISA: RV32I — complete base integer instruction set only

Configuration Parameters

.ENABLE_COUNTERS	(1)	cycle/instret counters ON
.ENABLE_REGS_16_31	(1)	full 32-register file
.ENABLE_MUL	(0)	NO multiply (saves area)
.ENABLE_DIV	(0)	NO divide
.COMPRESSED_ISA	(0)	NO 16-bit instructions
.CATCH_ILLINSN	(1)	trap on illegal instruction
.CATCH_MISALIGN	(1)	trap on misaligned access
.PROGADDR_RESET	(0x0000_0000)	boot addr = ROM[0]
.STACKADDR	(0x0001_00FC)	stack top

Key I/O Ports

OUTPUT	mem_valid	Request pending
OUTPUT	mem_instr	1=fetch 0=data
OUTPUT	mem_addr[31:0]	Target address
OUTPUT	mem_wdata[31:0]	Write data
OUTPUT	mem_wstrb[3:0]	Byte enable
INPUT	mem_ready	Transaction OK
INPUT	mem_rdata[31:0]	Read data
INPUT	resetn	Active-low reset

RISC-V Registers (x0-x31)

Analogy: Registers are like your desk. RAM is the filing cabinet. Disk is the warehouse. You work with what's on your desk — fast access, but limited space (only 32 slots!).

x0 (zero)

Always reads as 0. Writes are ignored.

x8 (s0/fp)

Saved register / frame pointer.

x1 (ra)

Return address — where to go back after a function call.

x9 (s1)

Saved register, preserved across function calls.

x2 (sp)

Stack pointer — tracks the top of the call stack.

x10–x11 (a0–a1)

Function arguments 1 & 2, also return values.

x3 (gp)

Global pointer — base address of global variables.

x12–x17 (a2–a7)

Additional function arguments (up to 8 total).

x4 (tp)

Thread pointer — used in multi-threaded programs.

x18–x27 (s2–s11)

Saved registers — callee must restore if used.

x5–x7 (t0–t2)

Temporaries — scratch registers, not preserved across calls.

x28–x31 (t3–t6)

More temporaries — fast scratch space.

RISC-V Instruction Formats

All instructions are exactly 32 bits wide. The bits are arranged in 6 different patterns (formats) depending on the instruction type.

R-type

Register-to-register ops (ADD, SUB, AND...)

funct7 | rs2 | rs1 | funct3 | rd | opcode

I-type

Immediate arithmetic & loads (ADDI, LW...)

imm[11:0] | rs1 | funct3 | rd | opcode

S-type

Store to memory (SW, SB, SH...)

imm[11:5] | rs2 | rs1 | funct3 | imm[4:0] | opcode

B-type

Conditional branches (BEQ, BNE, BLT...)

imm[12|10:5] | rs2 | rs1 | funct3 | imm[4:1|11] | opcode

U-type

Upper immediate (LUI, AUIPC)

imm[31:12] | rd | opcode

J-type

Jump and link (JAL)

imm[20|10:1|11|19:12] | rd | opcode

Instruction Types & PicoRV32 Config Flags

Instruction Support

✓	Integer Arithmetic	ADD, SUB, ADDI, LUI, AUIPC
✓	Logic	AND, OR, XOR, ANDI, ORI, XORI
✓	Shift	SLL, SRL, SRA, SLLI, SRLI, SRAI
✓	Compare	SLT, SLTU, SLTI, SLTIU
✓	Loads / Stores	LW, LH, LB, SW, SH, SB
✓	Branches	BEQ, BNE, BLT, BGE, BLTU, BGEU
✓	Jumps	JAL, JALR
✗	Multiply (M ext.)	MUL, MULH, DIV, REM
✗	Compressed (C ext.)	16-bit encodings

Project-Specific Notes

Config Flags

ENABLE_MUL=0, ENABLE_DIV=0
COMPRESSED_ISA=0
BARREL_SHIFTER=0

First Instruction

0x00020137 = LUI x2, 0x20
→ Sets up stack pointer at boot

Compiler Flag

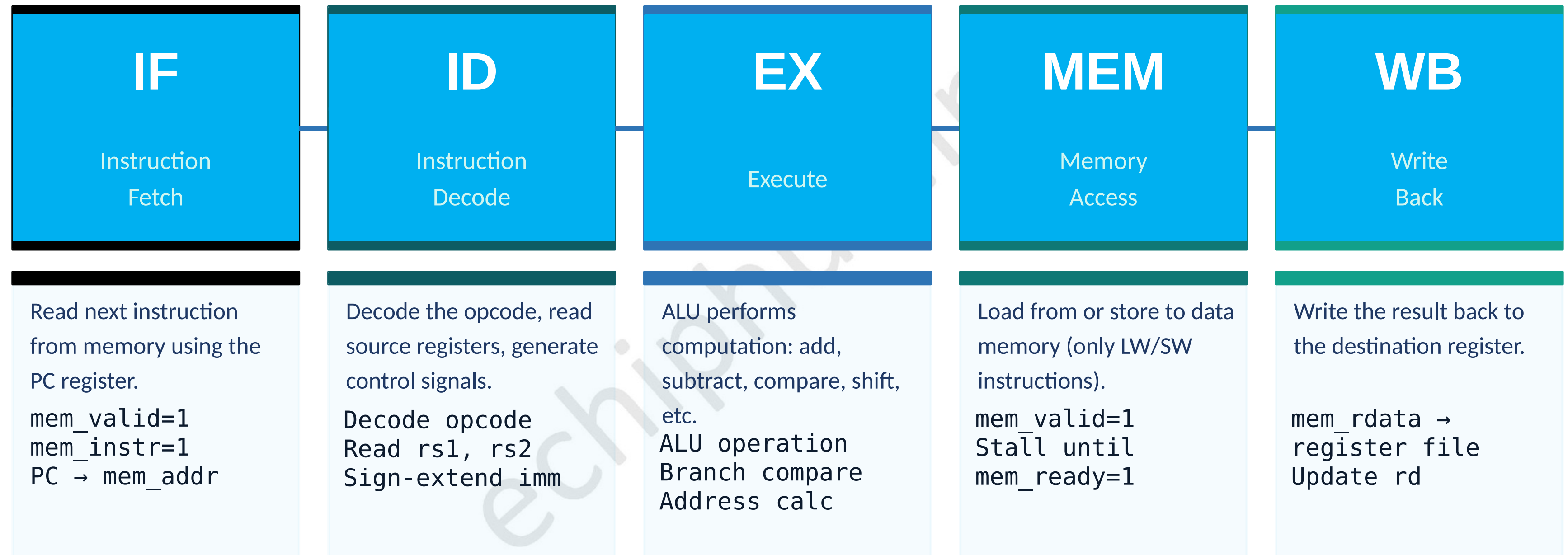
-march=rv32i -mabi=ilp32
Matches ENABLE_MUL=0 exactly

mem_instr Signal

=1 → instruction fetch
=0 → data access (LW/SW)

CPU Pipeline: IF / ID / EX / MEM / WB

Pipelining lets the CPU work on multiple instructions at the same time — like an assembly line.

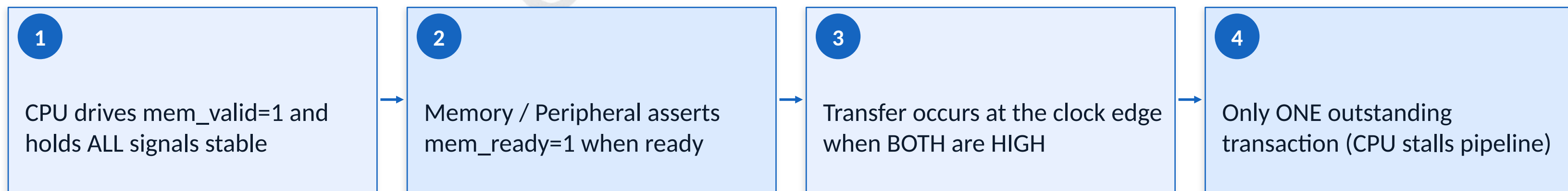


⚠ PicoRV32 Stall Rule: CPU holds `mem_valid=1` and freezes ALL signals until peripheral asserts `mem_ready=1` (one cycle pulse). Only ONE outstanding transaction at a time — no out-of-order execution.

PicoRV32 Native Memory Interface

Signal	Direction	Width	Description
mem_valid	CPU→MEM	1	Request pending — hold HIGH until mem_ready
mem_instr	CPU→MEM	1	1 = instruction fetch, 0 = data access
mem_addr	CPU→MEM	32	Target address
mem_wdata	CPU→MEM	32	Write data (stores only)
mem_wstrb	CPU→MEM	4	Byte write enable: 0=read, 0xF=32-bit write
mem_ready	MEM→CPU	1	Transaction accepted (one-cycle pulse)
mem_rdata	MEM→CPU	32	Read data — valid same cycle as mem_ready

Handshake Rule



Branch Hazards & PicoRV32 Trap Behavior

TWO_STAGE_SHIFT · CATCH_ILLINSN · trap Signal

Branch Hazards (General)

- Branch outcome not known until EX stage
- Naïve pipeline fetches next sequential instruction
- If branch taken → fetched instruction must be flushed
- Flush = wasted cycle (1 pipeline bubble for PicoRV32)
- PicoRV32 resolves branches in EX — simple stall, no speculation

PicoRV32-Specific Behavior

TWO_STAGE_SHIFT=1

Area optimization: shift ops take 2 cycles instead of 1. Saves LUTs in FPGA/ASIC implementation.

CATCH_ILLINSN=1

Any undefined opcode raises the trap signal. Firmware can detect and handle CPU halt.

CATCH_MISALIGN=1

Unaligned LW/SW (addr not multiple of 4) raises trap — important for AXI peripheral access.

trap Signal

Goes HIGH when CPU hits illegal instruction or misaligned access. Not exposed at top-level in current design but observable in testbench.

Lab: make pico — Run PicoRV32 Simulation

1

Navigate to project root

```
$ cd /path/to/picorv32_soc
```

Ensure Makefile is present in this directory

2

Run standalone PicoRV32 simulation

```
$ make pico
```

Builds firmware → Verilator compiles RTL → runs ./obj_pico/sim_pico → produces tb_picorv32.vcd

3

Open waveform in GTKWave

```
$ gtkwave tb_picorv32.vcd & OR: make wave
```

Add signals manually or load saved .gtkw config

Signals to ADD in GTKWave: clk • resetn • mem_valid • mem_instr • mem_ready • mem_addr[31:0] • mem_rdata[31:0]

RISC-V Ecosystem → THIS Project

THIS PROJECT

Simulation: Verilator

PnR: OpenLane

Synthesized: Yosys

SkyWater 130nm PDK

PicoRV32 + AXI-Lite + UART SoC on SkyWater 130nm PDK

`$ make pico`

→ Run PicoRV32 standalone simulation → tb_picorv32.vcd

`$ make synth`

→ Synthesize SoC with Yosys → gate-level netlist
Full physical design: floorplan → routing → GDS-II

`$ make wave`

→ Open waveform: gtkwave tb_picorv32.vcd

Broader RISC-V domains: IoT, AI Edge, Linux servers, automotive, space — our SoC Subsystem is the hands-on proof-of-concept for this course.

Key Takeaways

PicoRV32 is RV32I

32 registers, fixed 32-bit instruction width, in-order execution. No mul/div/float in this project.

Boot & Stack

PC starts at 0x0000_0000 → First instruction: LUI x2, 0x20 (stack init). Stack top: 0x0001_00FC.

Run the Lab

make pico → Verilator simulation → tb_picorv32.vcd → GTKWave.

Native Bus Protocol

mem_valid + mem_ready — simple stall protocol. CPU holds all signals until peripheral responds.

Config Matters

ENABLE_MUL=0, ENABLE_DIV=0, COMPRESSED_ISA=0. Compiler must match: -march=rv32i -mabi=ilp32.

Watch the Waveform

mem_addr increments 0x0→0x4→0x8. mem_instr=1 on every fetch. First rdata=0x00020137.